

AI-Powered Credit Risk Intelligence Platform

NeoStats AI Engineer Assignment — Submission

- Dataset: Home Credit Default Risk (Kaggle)
- Stack: Python 3.11, LightGBM, SHAP, Streamlit, SQLite, Docker
- LLM: Multi-provider (OpenAI / Groq / Gemini) + deterministic fallback
- Author: Ashwani Dhayal

Business Problem

- Banks must make faster, more accurate, and explainable credit decisions.
- Identify high-risk applicants early and automate risk scoring.
- Provide auditable, regulator-friendly reasons for every decision.
- Let business analysts explore the portfolio in plain English.
- Bridge ML insights and credit policy through readable rules.

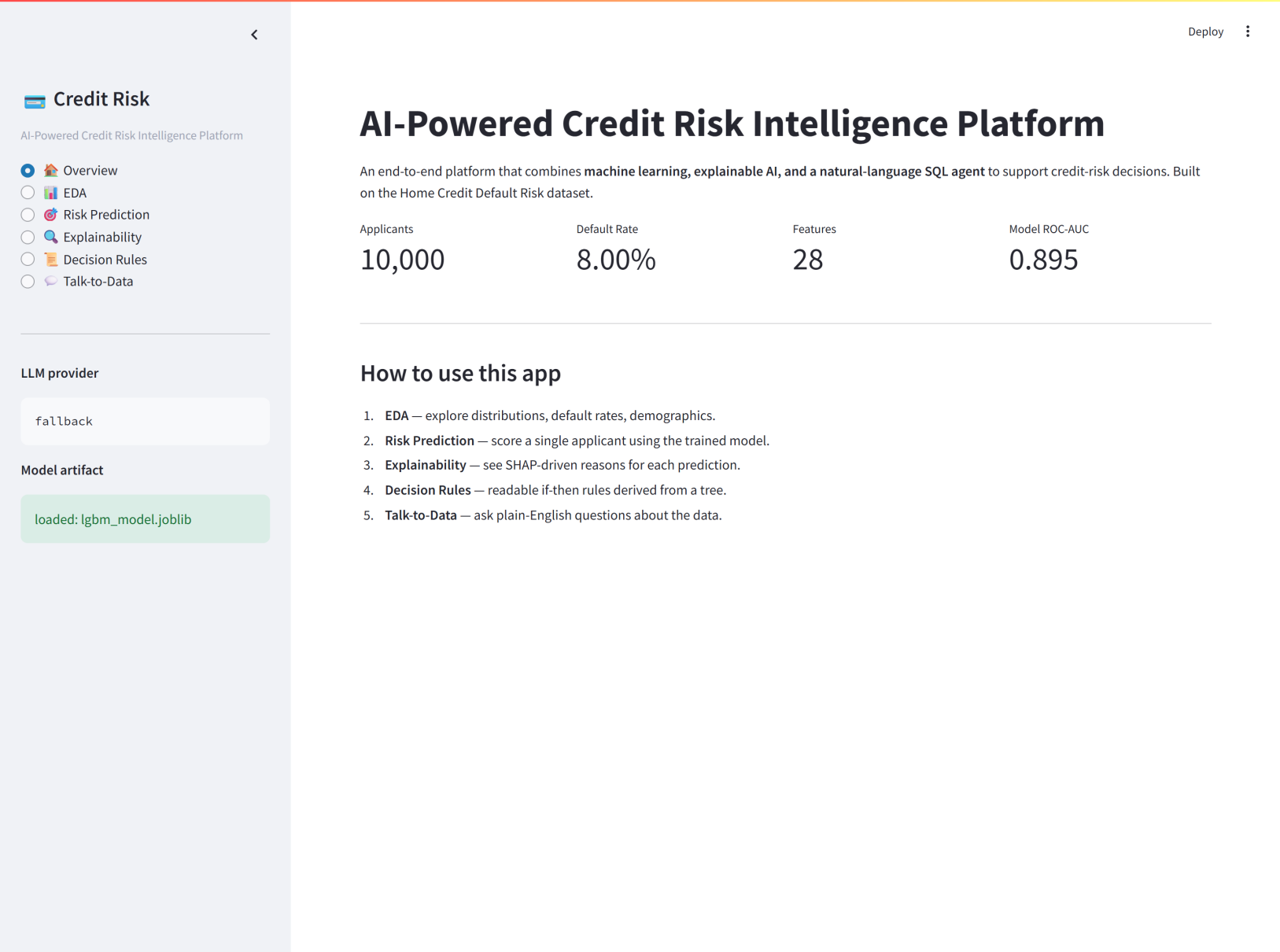
Solution at a Glance

- End-to-end platform: EDA → ML → Explainability → Rules → Chatbot.
- Single Streamlit UI with 6 sections; one Docker command to run.
- ML model returns probability + Low/Medium/High band + Decision.
- SHAP TreeExplainer attributes every score to ranked feature drivers.
- Talk-to-Data agent translates English to safe, read-only SQLite SQL.
- Decision-tree rules give credit officers an audit-ready policy view.

Architecture (logical view)

- Streamlit UI (app/streamlit_app.py) — multi-section navigation.
- src/ml: LightGBM training / inference / SHAP TreeExplainer.
- src/rules: depth-4 decision tree → human-readable IF-THEN rules.
- src/data: schema, sample generator, CSV → SQLite ingestion.
- src/llm: OpenAI / Groq / Gemini auto-detect + safe NL→SQL agent.
- src/utis/sql_safety: read-only SELECT guardrails (no DDL/DML).
- Single SQLite DB is the source of truth for ML and the chatbot.

UI - Overview Section



Live KPIs of the platform: 10,000 applicants, 8% default rate, 28 features after engineering, model ROC-AUC 0.895. The sidebar shows the active LLM provider and the loaded model artifact.

UI - Exploratory Data Analysis

Credit Risk

AI-Powered Credit Risk Intelligence Platform

Overview

EDA

Risk Prediction

Explainability

Decision Rules

Talk-to-Data

LLM provider

fallback

Model artifact

loaded: lgbm_model.joblib

Deploy

Exploratory Data Analysis

Summary

Demographics

Financials

Default Drivers

Dataset summary

Rows: 10,000

Columns: 30

Default rate: 8.00%

Missing values (top 10):

column	missing_count	missing_pct
EXT_SOURCE_1	5,504	55.04
OCCUPATION_TYPE	3,157	31.57
EMPLOYMENT_AGE_RATIO	1,782	17.82
EMPLOYMENT_YEARS	1,782	17.82
DAYS_EMPLOYED	1,782	17.82
SK_ID_CURR	0	0
TARGET	0	0
CREDIT_TERM	0	0
ANNUITY_INCOME_RATIO	0	0
CREDIT_INCOME_RATIO	0	0

Sample rows

	SK_ID_CURR	TARGET	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	AMT_ANNUITY
0	100,001	0	Revolving loans	M	N	Y	0	155,500	15,500
1	100,002	0	Cash loans	F	N	N	1	226,300	22,630
2	100,003	0	Cash loans	M	N	Y	0	145,200	14,520
3	100,004	0	Cash loans	M	Y	Y	0	131,900	13,190
4	100,005	0	Cash loans	F	Y	N	0	152,700	15,270
5	100,006	0	Cash loans	M	N	N	1	157,300	15,730
6	100,007	0	Cash loans	M	Y	N	0	68,600	6,860
7	100,008	0	Cash loans	M	Y	N	0	256,200	25,620

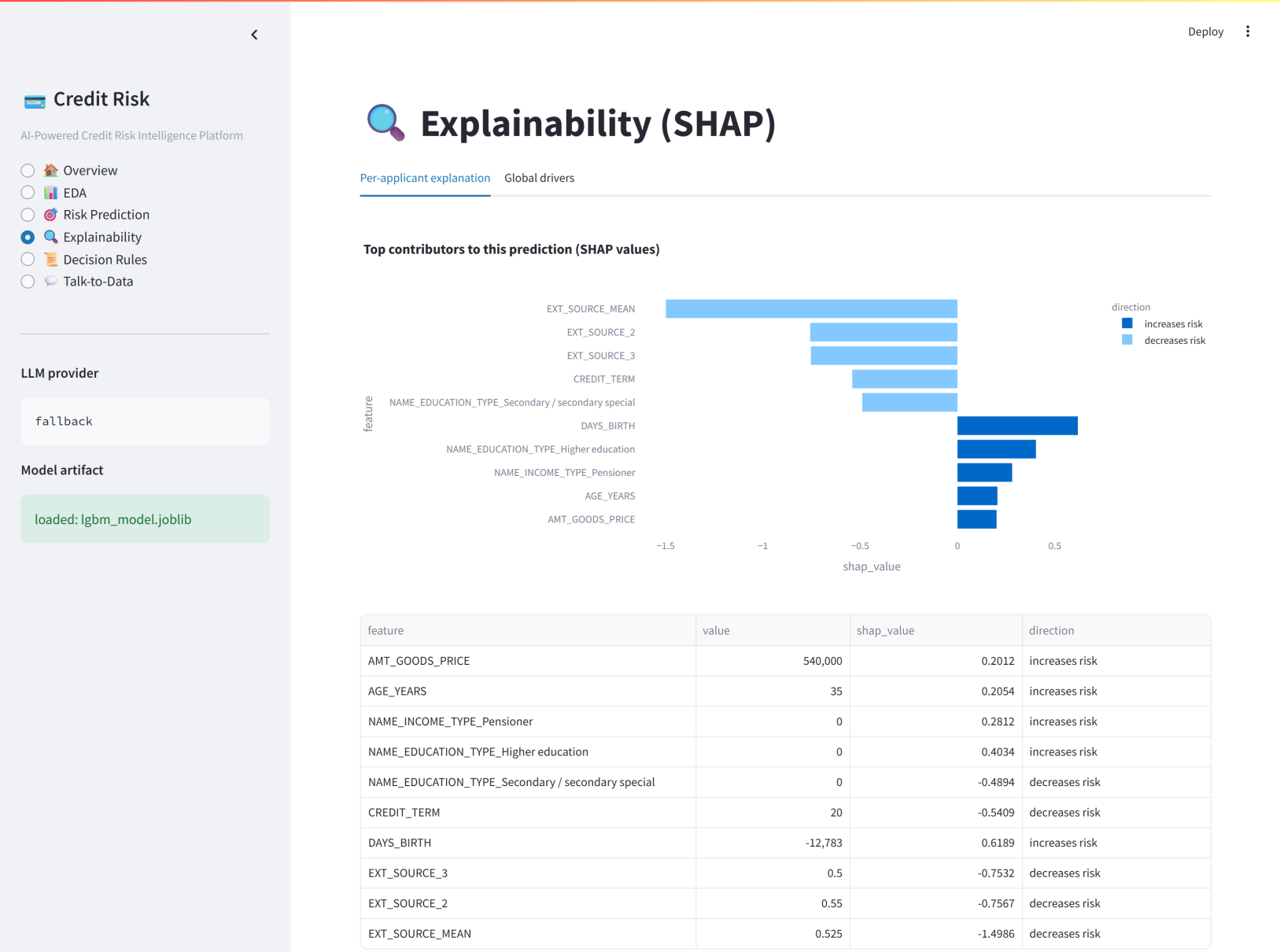
Four EDA tabs: Summary, Demographics, Financials, Default Drivers. Distributions are split by default status; the table on the right shows missingness for the most-affected columns.

UI - Risk Prediction

[illegible]

Form-driven applicant scoring. Output: probability of default + risk band (Low / Medium / High) + suggested decision (Approve / Review / Reject). Sample shown: $P(\text{default})=0.06\% \rightarrow \text{Low} \rightarrow \text{Approve}$.

UI · Explainability (SHAP)



UI · Decision Rules

Credit Risk

AI-Powered Credit Risk Intelligence Platform

Overview

EDA

Risk Prediction

Explainability

Decision Rules

Talk-to-Data

LLM provider

fallback

Model artifact

loaded: lgbm_model.joblib

Deploy

Business-Readable Decision Rules

Rules are derived from a depth-limited decision tree fit on the same engineered features as the LightGBM model. Each row is a leaf — the table shows its support, default rate, and lift vs. the base rate.

rule_id	band	conditions	support_pct	default_rate_pct	lift	n_samples
1	High	CREDIT_INCOME_RATIO <= 5.188 AND EXT_SOURCE_MEAN <= 0.292 AND NAME_EDUC	11.79	66.72	8.34	1,179
13	High	CREDIT_INCOME_RATIO > 5.188 AND ANNUITY_INCOME_RATIO > 0.458 AND EXT_SOU	6.86	88.31	11.04	686
8	Medium	CREDIT_INCOME_RATIO <= 5.188 AND EXT_SOURCE_MEAN > 0.292 AND CREDIT_INCO	17.3	24.71	3.09	1,730
7	High	CREDIT_INCOME_RATIO <= 5.188 AND EXT_SOURCE_MEAN > 0.292 AND CREDIT_INCO	5.79	55.18	6.9	579
9	High	CREDIT_INCOME_RATIO > 5.188 AND ANNUITY_INCOME_RATIO <= 0.458 AND EXT_SOU	2.68	79.95	9.99	268
6	Medium	CREDIT_INCOME_RATIO <= 5.188 AND EXT_SOURCE_MEAN > 0.292 AND CREDIT_INCO	6.31	29.35	3.67	631
15	High	CREDIT_INCOME_RATIO > 5.188 AND ANNUITY_INCOME_RATIO > 0.458 AND EXT_SOU	2.47	61.39	7.67	247
14	High	CREDIT_INCOME_RATIO > 5.188 AND ANNUITY_INCOME_RATIO > 0.458 AND EXT_SOU	2	71.63	8.95	200
12	Medium	CREDIT_INCOME_RATIO > 5.188 AND ANNUITY_INCOME_RATIO <= 0.458 AND EXT_SOU	2.66	45.49	5.69	266
5	Low	CREDIT_INCOME_RATIO <= 5.188 AND EXT_SOURCE_MEAN > 0.292 AND CREDIT_INCO	31.01	11.01	1.38	3,101

Rule cards

Rule 1 — High risk · support 11.79% · default rate 66.72% · lift 8.34×

IF CREDIT_INCOME_RATIO <= 5.188 AND EXT_SOURCE_MEAN <= 0.292 AND NAME_EDUCATION_TYPE != 'Higher education' AND

Rule 13 — High risk · support 6.86% · default rate 88.31% · lift 11.04×

IF CREDIT_INCOME_RATIO > 5.188 AND ANNUITY_INCOME_RATIO > 0.458 AND EXT_SOURCE_MEAN <= 0.471 AND NAME_EDUCATION.

Rule 8 — Medium risk · support 17.3% · default rate 24.71% · lift 3.09×

Business-readable IF-THEN rules derived from a depth-4 decision tree. Each row reports support (% of population), default rate inside the leaf, and lift vs. the 8% base rate.

UI · Talk-to-Data Chatbot

Credit Risk

AI-Powered Credit Risk Intelligence Platform

Overview

EDA

Risk Prediction

Explainability

Decision Rules

Talk-to-Data

LLM provider

fallback

Model artifact

loaded: lgbm_model.joblib

Deploy

Talk-to-Data

Active LLM provider: fallback (set OPENAI_API_KEY / GROQ_API_KEY / GEMINI_API_KEY to switch)

Try a sample question:

How many applicants are there in total?

What is the overall default rate?

Default rate by education level

Top 5 occupations by default rate (min 100 clients)

Average income for defaulters vs non-defaulters

Default rate by gender

Ask a question about the data:

How many applicants are there in total?

Ask

Total Applicants: 10000

Generated SQL

Result rows

	total_applicants
0	10,000

Provider: fallback (deterministic fallback)

User asks a plain-English question; the agent emits a safe `SELECT`, runs it on `SQLite`, and summarises the rows. The generated `SQL` is exposed for full transparency. Falls back to a deterministic intent parser when no LLM key is set.

Backend · ML Training (src/ml/train.py)

LightGBM with scale_pos_weight for class imbalance, threshold chosen by Youden's J on the validation set.

```
pos_weight = float((y_train == 0).sum() / max((y_train == 1).sum(), 1))

lgbm = LGBMClassifier(
    n_estimators=400,
    learning_rate=0.05,
    num_leaves=31,
    max_depth=-1,
    min_child_samples=40,
    reg_alpha=0.1,
    reg_lambda=0.1,
    subsample=0.85,
    colsample_bytree=0.85,
    scale_pos_weight=pos_weight,
    random_state=random_state,
    n_jobs=-1,
    verbose=-1,
)

pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", lgbm),
])
pipeline.fit(X_train, y_train)

proba_val = pipeline.predict_proba(X_val)[:, 1]
auc = float(roc_auc_score(y_val, proba_val))
pr_auc = float(average_precision_score(y_val, proba_val))
ks = _ks_statistic(y_val, proba_val)

# Pick the threshold that maximises Youden's J on the validation set.
fpr, tpr, thresholds = roc_curve(y_val, proba_val)
j = tpr - fpr
best_idx = int(np.argmax(j))
best_threshold = float(thresholds[best_idx])
preds = (proba_val >= best_threshold).astype(int)
f1 = float(f1_score(y_val, preds))
cm = confusion_matrix(y_val, preds).tolist()

feat_names = get_feature_names_out(pipeline.named_steps["preprocessor"])
importances_arr = pipeline.named_steps["classifier"].feature_importances_
importances = dict(
```

Backend · NL → SQL Agent (src/llm/nl_to_sql.py)

Pipeline: prompt LLM → parse JSON → static safety check → read-only execute → second LLM call grounds the summary in rows.

```
sql_raw = _fallback_sql(question) or (
    "SELECT 'unanswerable_with_fallback_parser' AS note"
)
else:
    try:
        client = get_client()
        llm_text = client.chat(build_messages(schema, question))
        sql_raw = _extract_json_sql(llm_text)
    except (LLMUnavailable, Exception):
        # LLM died -- use the deterministic parser instead of giving up.
        used_fallback = True
        provider = "fallback"
        sql_raw = _fallback_sql(question) or (
            "SELECT 'llm_failed_and_no_fallback_match' AS note"
        )

validation = validate_and_harden(sql_raw)
if not validation.ok:
    return AgentAnswer(
        question=question,
        sql=sql_raw,
        rows=[],
        answer=f"I rejected the generated SQL: {validation.reason}",
        provider=provider,
        used_fallback=used_fallback,
        error=validation.reason,
    )

try:
    with open_connection() as conn:
        df = pd.read_sql_query(validation.sql, conn)
except sqlite3.Error as e:
    return AgentAnswer(
        question=question,
        sql=validation.sql,
        rows=[],
        answer=f"SQL execution failed: {e}",
        provider=provider,
        used_fallback=used_fallback,
        error=str(e),
    )
```

```
)  
  
summary = _summarise(question, validation.sql, df, provider)  
return AgentAnswer(  
    question=question,  
    sql=validation.sql,  
    rows=df.to_dict(orient="records"),  
    answer=summary,  
    provider=provider,  
    used_fallback=used_fallback,  
)
```

Backend · SQL Safety (src/utils/sql_safety.py)

Single SELECT, table allowlist, blocked keyword list, automatic LIMIT cap. The chatbot cannot mutate data even if the LLM goes off-script.

```
    r"\bpragma\b",
    r"\bvacuum\b",
    r"\bload_extension\b",
    r"\b\.shell\b",
    r"\b\.system\b",
]
MAX_ROWS = 200

@dataclass
class SqlValidation:
    ok: bool
    sql: str
    reason: str = ""

def validate_and_harden(sql):
    raw = sql.strip().rstrip(";").strip()
    if not raw:
        return SqlValidation(False, "", "Empty SQL.")

    if ";" in raw:
        return SqlValidation(False, raw, "Multiple statements are not allowed.")

    lowered = raw.lower()
    for pat in FORBIDDEN_PATTERNS:
        if re.search(pat, lowered):
            return SqlValidation(False, raw, f"Forbidden keyword detected ({pat}).")

    parsed = sqlparse.parse(raw)
    if len(parsed) != 1:
        return SqlValidation(False, raw, "Exactly one statement is required.")

    first_token = parsed[0].token_first(skip_ws=True, skip_cm=True)
    if first_token is None:
        return SqlValidation(False, raw, "Could not parse SQL.")

    head = first_token.normalized.upper()
    if head not in {"SELECT", "WITH"}:
        return SqlValidation(False, raw, "Only SELECT/WITH queries are permitted.")
```

```
# Recognise CTE aliases so `WITH t AS (...) SELECT * FROM t` works.
cte_aliases = set(
    re.findall(r"\b([a-zA-Z_][a-zA-Z0-9_]*)\s+as\s*\s*(\"", lowered)
)
table_refs = set(re.findall(r"\bfrom\s+([a-zA-Z_][a-zA-Z0-9_]*)", lowered))
table_refs |= set(re.findall(r"\bjoin\s+([a-zA-Z_][a-zA-Z0-9_]*)", lowered))
unknown = table_refs - ALLOWED_TABLES - cte_aliases
if unknown:
    return SqlValidation(
        False, raw, f"Reference to disallowed table(s): {sorted(unknown)}"
    )
```

Backend · Multi-Provider LLM Client (src/llm/client.py)

Auto-detects OpenAI / Groq / Gemini based on whichever API key is set; gracefully falls back to a deterministic parser.

```
from openai import OpenAI
self._client = OpenAI(
    api_key=SETTINGS.openai_api_key,
    timeout=SETTINGS.llm_timeout_seconds,
)

@retry(stop=stop_after_attempt(2), wait=wait_exponential_jitter(initial=1, max=4))
def chat(self, messages):
    resp = self._client.chat.completions.create(
        model=SETTINGS.openai_model,
        messages=messages,
        temperature=SETTINGS.llm_temperature,
        max_tokens=SETTINGS.llm_max_tokens,
    )
    return (resp.choices[0].message.content or "").strip()

class _GroqClient(_BaseClient):
    name = "groq"

    def __init__(self):
        from groq import Groq
        self._client = Groq(
            api_key=SETTINGS.groq_api_key,
            timeout=SETTINGS.llm_timeout_seconds,
        )

    @retry(stop=stop_after_attempt(2), wait=wait_exponential_jitter(initial=1, max=4))
    def chat(self, messages):
        resp = self._client.chat.completions.create(
            model=SETTINGS.groq_model,
            messages=messages,
            temperature=SETTINGS.llm_temperature,
            max_tokens=SETTINGS.llm_max_tokens,
        )
        return (resp.choices[0].message.content or "").strip()

class _GeminiClient(_BaseClient):
    name = "gemini"
    def __init__(self):
```



```

import google.generativeai as genai
genai.configure(api_key=SETTINGS.gemini_api_key)
self._model = genai.GenerativeModel(SETTINGS.gemini_model)

@retry(stop=stop_after_attempt(2), wait=wait_exponential_jitter(initial=1, max=4))
def chat(self, messages):
    # Gemini has no system role - merge into the user prompt.
    sys_parts = [m["content"] for m in messages if m["role"] == "system"]
    usr_parts = [m["content"] for m in messages if m["role"] != "system"]
    prompt = ("\n\n".join(sys_parts) + "\n\n" + "\n\n".join(usr_parts)).strip()
    resp = self._model.generate_content(
        prompt,
        generation_config={
            "temperature": SETTINGS.llm_temperature,
            "max_output_tokens": SETTINGS.llm_max_tokens,
        },
    )

```

Prompt Engineering & Token Optimisation

- JSON-only output contract — model emits {"sql": "..."}; deterministic parsing.
- Pinned schema (~25 cols, 1-line descriptions) → no DB introspection round-trip.
- Few-shot kept tiny (4 examples) to anchor style without bloating context.
- Temperature 0; max 256 output tokens — SQL doesn't need prose.
- Static SQL safety: SELECT/WITH only, table allowlist, LIMIT cap, blocked keywords.
- Result-grounded summary prompt explicitly forbids invented numbers.
- Hard fallback: deterministic regex parser keeps the demo working offline.

Evaluation Results (bundled 10k sample)

Metric	Value
ROC-AUC	0.8952
PR-AUC	0.5109
KS Statistic	0.6571
F1 @ optimal threshold	0.3994
Optimal threshold	0.0631
Default rate	0.08
Train rows / Val rows	8000 / 2000

Top Derived Decision Rules

- R1 · High · support 11.79% · default rate 66.72% · lift 8.34×
- R13 · High · support 6.86% · default rate 88.31% · lift 11.04×
- R8 · Medium · support 17.3% · default rate 24.71% · lift 3.09×
- R7 · High · support 5.79% · default rate 55.18% · lift 6.9×
- R9 · High · support 2.68% · default rate 79.95% · lift 9.99×

How To Run

- `git clone https://github.com/ashwani-dhayal/credit-risk-platform.git`
- `cd credit-risk-platform`
- `cp .env.example .env` # add ONE LLM key (optional)
- `docker compose up --build`
- → open `http://localhost:8501`
- Local: `python -m venv .venv ; pip install -r requirements.txt ; python scripts/train_model.py ; streamlit run app/streamlit_app.py`

Limitations & Next Steps

- Adding bureau / previous_application aggregates would lift AUC $\sim +0.03$.
- No data-drift monitoring yet (Evidently / WhyLogs are obvious next steps).
- Single-tenant SQLite — switch to Postgres for multi-user concurrency.
- No UI auth — deploy behind a reverse proxy with auth in production.
- LLM cost cap is per-call only; daily quotas would need Redis-based limiting.

Thank You

- Repo: <https://github.com/ashwani-dhayal/credit-risk-platform>
- Author: Ashwani Dhayal
- Submission: NeoStats AI Engineer assignment.